

DOSSIER TECHNIQUE DE LIVRAISON — OAF Connect (One Africa Forums)

Document d'archivage et de transmission. Dernière mise à jour : 13 juin 2026.

But : pouvoir **archiver tout le projet**, le **reprendre par n'importe quel développeur**, et ne **jamais dépendre d'un seul environnement ou d'une seule personne**.

0) STATUT AU 13 JUIN 2026 (état réel vérifié)

Légende : confirmé / à confirmer dans le tableau de bord concerné (info non visible depuis le code).

Élément	Statut	Détail
Supabase — plan	Pro	Backups quotidiens + PITR 7 jours actifs . Capacité temps réel relevée.
Supabase — organisation	OneAfricaForums	Projet 0af-project-prod (ref qxxjsqctkltnjptqdbye).
Domaine d'envoi e-mail (Resend)	Vérifié	send.oneafricaforums.com — SPF + DKIM valides (juin 2026).
Connexion Google (OAuth)	Publiée et fonctionnelle	—
Onboarding obligatoire (nom + fonction/société)	En production	—
Module Partenaires premium	En production	logos, niveaux, contacts, liens.
Programmes APIDE (Abidjan + Lomé) + intervenants + participants test	Importés	CSV dans docs/programmes/ .
Bouton LinkedIn	Masqué proprement	provider non configuré (réaffichable en 1 ligne).
Service Worker (cache)	oaf-connect-v52	à incrémenter à chaque évolution frontend.
Kit de diffusion (QR, affiche, textes, DMARC, IT)	Livré	docs/diffusion/ .
GitHub — propriété	À confirmer	Le dépôt pointe encore vers github.com/hsidqui-dotcom/Networking- . Vérifier s'il a été transféré vers une organisation au nom de l'entreprise (ou si le compte a été rattaché).
Resend — plan d'envoi	À confirmer	Vérifier le quota/jour vs le nombre total d'invitations des 2 forums (point critique avant l'événement).
Propriété Google / DNS Genious	À confirmer	Comptes au nom de l'entreprise + 2 admins.

Les lignes sont les seuls points ouverts. Tout le reste est en production et vérifié.

1) OÙ EST LE CODE SOURCE

- **Hébergement du code : GitHub.**
- **Dépôt (repository) :** `hsidqui-dotcom/Networking-` → URL : `https://github.com/hsidqui-dotcom/Networking-`
- **Branche déployée en production :** `claude/oneafricaforums-networking-app-hVvUm` (c'est cette branche qui est mise en ligne automatiquement — voir §3).
- **Tout le code est dans ce dépôt** (frontend, scripts base de données, docs, tests). Il n'y a **pas** de code ailleurs (pas de Vercel/Replit séparé).

À télécharger / conserver : sur GitHub → bouton « **Code** » → « **Download ZIP** » (ou `git clone`). C'est la **copie complète** du code source.

2) COPIE STRUCTURÉE DU CODE SOURCE

L'app est une **PWA statique (HTML/CSS/JavaScript)** — pas de build, pas de serveur applicatif à compiler. Le « backend » est **Supabase** (base + API + auth gérés).

```

Networking-/
├─ index.html          # Racine : redirige vers /app/ (préserve le jeton OAuth)
├─ CNAME               # Domaine personnalisé : app.oneafricaforums.com
├─ .github/workflows/
│   └─ pages.yml       # Déploiement automatique (GitHub Actions → GitHub Pages)
│
├─ app/                # ← L'APPLICATION (frontend + intégration Supabase)
│   ├─ index.html      # App participant (PWA)
│   ├─ admin.html      # Console organisateur
│   ├─ app.js           # Logique app participant
│   ├─ admin.js        # Logique console organisateur
│   ├─ store.js        # Miroir d'état local (localStorage)
│   ├─ supabase.js     # Couche d'authentification / connexion Supabase
│   ├─ config.js       # Configuration : URL Supabase + clé publique (anon)
│   ├─ sw.js           # Service Worker (PWA, cache réseau-d'abord)
│   ├─ style.css       # Styles
│   ├─ manifest.webmanifest # Manifeste PWA (installation sur téléphone)
│   └─ icon*.png / icon.svg # Icônes
│
├─ supabase/           # ← BASE DE DONNÉES (scripts SQL = "backend")
│   ├─ schema.sql      # Schéma principal (tables, RLS, triggers)
│   ├─ access-control.sql # Annuaire cloisonné (H1) + inscription (H2) + handle_new_user
│   ├─ security-hardening.sql, perf-p0.sql # Sécurité + index de performance
│   ├─ messaging-safety.sql # Blocage / anti-spam / signalements
│   ├─ sponsors-plus.sql # Partenaires enrichis + table sponsor_contacts
│   ├─ invitations.sql / invite-guests.sql / invite-all.sql # E-mails (Resend)
│   ├─ make-admin.sql, seed-demo.sql, add-event-*.sql, ... # Utilitaires
│   └─ README.md
│
├─ docs/               # Documentation & données
│   ├─ AUDIT-2026-06-10.md # Audit expert + feuille de route
│   ├─ LIVRABLE-TECHNIQUE.md # CE document
│   ├─ pilote-checklist.md, product-spec.md, ...
│   ├─ programmes/      # CSV prêts (programmes APIDE + intervenants + participants)
│   └─ diffusion/       # Kit d'accès prêt à diffuser (QR, affiche A5, textes, DMARC, IT)
│
└─ tests/              # Tests & diagnostics
    ├─ functional-checklist.md # Checklist de tests fonctionnels
    ├─ smoke.html, diag.html  # Diagnostics navigateur (latence, moteur de connexion)
    └─ stress/              # Harnais de test de charge (k6 + scripts staging)

```

3) HÉBERGEMENT & DÉPLOIEMENT

Élément	Détail
Hébergeur du site	GitHub Pages (gratuit)
Plateforme de déploiement	GitHub Actions → <code>.github/workflows/pages.yml</code>
Domaine connecté	<code>app.oneafricaforums.com</code> (fichier <code>CNAME</code> , HTTPS forcé)
App participant	<code>https://app.oneafricaforums.com/app/</code>
Console organisateur	<code>https://app.oneafricaforums.com/app/admin.html</code>
Backend / base / API	Supabase Pro (projet <code>Oaf-project-prod</code> , ref <code>qxxjsqctkltnjptqbye</code> , org OneAfricaForums)
E-mails	Resend (domaine vérifié <code>send.oneafricaforums.com</code>)

Comment ça se redéploie (automatique)

À **chaque** `git push` sur la branche `claude/oneafricaforums-networking-app-hVvUm` , l'action GitHub publie **tout le dépôt** sur GitHub Pages (≈ 1 min). **Aucune commande manuelle.**

Redéployer en cas de bug / perte d'accès / changement de prestataire

1. **Bug** : corriger le code → `git commit` → `git push` sur la branche → l'action redéploie. (Ou « Re-run » la dernière action dans GitHub → Actions.)
2. **Repartir de zéro / autre hébergeur** : le site étant **100 % statique**, il peut être servi **n'importe où** (Netlify, Vercel, Cloudflare Pages, un simple serveur web) → il suffit d'uploader le contenu du dépôt et de pointer le domaine. Seule contrainte : garder `app/config.js` (clés Supabase) et le fichier `CNAME` .
3. **Domaine** : géré chez ton registrar DNS (voir §6). Le sous-domaine `app` pointe vers GitHub Pages.

4) STRUCTURE TECHNIQUE

Architecture générale

Client lourd statique + Backend managé (Jamstack). Le navigateur charge l'app (HTML/JS) depuis GitHub Pages, puis parle **directement** à Supabase (base Postgres + Auth + Realtime) via la clé publique `anon` , sécurisé par **RLS** (Row Level Security) côté base. Pas de serveur intermédiaire à maintenir.

```

[Navigateur / PWA]  ≈  [Supabase : Postgres + Auth + Realtime]
    |                      |
    |                      |
GitHub Pages        Resend (e-mails) via pg_net + Vault
(fichiers statiques)  Google OAuth (connexion)

```

Principales fonctionnalités

- **Multi-événements (forums)**, programme bilingue FR/EN, intervenants, **partenaires premium** (logo, niveau, contacts, liens).
- **Annuaire participants** cloisonné par forum + **matching** par affinités.
- **Connexion** : Google + code e-mail (OTP). **Onboarding obligatoire** (nom + fonction/société).
- **Messagerie 1:1 temps réel**, blocage, signalement, anti-spam.
- **Connexions / mise en relation, rendez-vous 1:1, Q&A en direct, notifications.**
- **Console organisateur** : événements, programme (CSV), intervenants, **participants (import + ajout/édition manuelle)**, partenaires, notifications, **invitations e-mail (1 clic)**, modération, QR code.
- **PWA installable** (iPhone/Android), fonctionne hors-ligne (cache).

Technologies

- **Frontend** : HTML5, CSS3, **JavaScript vanilla** (aucun framework, aucun build).
- **PWA** : Service Worker + manifest.
- **Backend (BaaS)** : **Supabase** — PostgreSQL, Auth (OTP + OAuth), Realtime, Vault (secrets), extension **pg_net** (appels HTTP sortants).
- **E-mails** : **Resend** (SMTP pour les codes + API HTTP pour les invitations).
- **Librairies chargées à l'exécution depuis le CDN esm.sh** : `@supabase/supabase-js` (auth/temps réel) et `qrcode` (console).
- **Hébergement** : GitHub Pages + GitHub Actions.

Dépendances importantes

Dépendance	Rôle	Où
<code>@supabase/supabase-js@2</code>	Connexion base/auth/temps réel	chargé depuis <code>https://esm.sh</code> à l'exécution
<code>qrcode@1.5.3</code>	Génération du QR (console)	<code>https://esm.sh</code>
pg_net (Postgres)	Envoi des e-mails d'invitation	extension Supabase
> Ces 2 libs JS sont chargées en direct depuis esm.sh . Voir §8 (risque + recommandation de "vendoring").		

Fonctionnement de la base de données (tables clés)

`profiles` (utilisateurs), `events` (forums), `sessions`, `speakers`, `session_speakers`, `sponsors` + `sponsor_contacts`, `event_attendees` (qui est dans quel forum), `connections`, `messages`, `meetings`, `bookmarks`, `notifications`, `guests` (participants importés "à réclamer"), `questions` / `question_votes` (Q&A), `blocks`, `reports`, `invites`. **Sécurité** : RLS activée partout ; fonctions clés `handle_new_user` (création de profil + inscription), `is_admin`, `shares_event` (cloisonnement), `send_event_invites` / `invite_event_guests` (e-mails).

5) BASE DE DONNÉES & SAUVEGARDES

- **Où sont les données** : Supabase, projet `oaf-project-prod` (PostgreSQL managé). Région : celle choisie à la création du projet.
 - **Exporter la base** :
 - Supabase Dashboard → **Database** → **Backups** (selon le plan) ; **ou**
 - **Project Settings** → **Database** → **Connection string** puis `pg_dump` : `pg_dump "postgresql://postgres:[MDP]@db.qxxjsqctkltnjptqbye.supabase.co:5432/postgres" -Fc -f oaf_backup.dump`
 - Export rapide d'une table en CSV : Table Editor → **Export**.
 - **Restaurer** : sur un projet Supabase neuf → exécuter les fichiers `supabase/*.sql` (voir ordre dans `docs/AUDIT-2026-06-10.md`), puis `pg_restore` du dump, ou réimporter les CSV.
 - **Procédures de backup** :
 - **Le projet est en Supabase Pro** : sauvegardes quotidiennes automatiques + **PITR (Point-in-Time Recovery) 7 jours** sont actives (Dashboard → Database → Backups).
 - **Bonne pratique complémentaire** (ceinture + bretelles) : faire un `pg_dump` **manuel avant et après chaque événement**, stocké hors-ligne.
 - Conserver aussi les **CSV sources** (déjà dans `docs/programmes/`).
-

6) ACCÈS INDISPENSABLES À CONSERVER

Stocker ces accès dans un **gestionnaire de mots de passe** (1Password/Bitwarden) **au nom de l'entreprise**. Ne jamais les mettre en clair dans le code.

Service	Usage	À conserver
GitHub (dépôt <code>hsidqui-dotcom/Networking-</code>)	Code source + déploiement	Identifiants + droits sur le dépôt. Confirmer que le dépôt est rattaché à une organisation au nom de l'entreprise (voir §0).
Supabase Pro (org OneAfricaForums, projet <code>Oaf-project-prod</code>)	Base, auth, temps réel, backups	Login + URL projet + clés (anon publique, service_role secrète)
Resend	E-mails	Login + clé API re_... (aussi dans Supabase Vault <code>resend_api_key</code>)
Google Cloud Console	Connexion Google (OAuth)	Login + Client ID / Client Secret OAuth
Registrar du domaine	<code>oneafricaforums.com</code> (DNS)	Accès DNS (sous-domaines <code>app</code> et <code>send</code>)
Microsoft 365 / messagerie	Réception des e-mails de test	—

Variables / clés (emplacement, sans exposer les secrets)

- **Publiques (déjà dans le code, normal)** : `app/config.js` → `SUPABASE_URL` + `SUPABASE_ANON_KEY` (clé publishable, sans danger).
- **Secrètes (jamais dans le code)** :
- **Resend API key** → stockée dans **Supabase** → **Vault** (`resend_api_key`) **et** dans **Auth** → **SMTP Settings**.
- **Google Client Secret** → **Supabase** → **Auth** → **Providers** → **Google**.
- **Supabase `service_role`** → **uniquement** pour des scripts d'admin/staging (jamais côté client).

7) GUIDE DE REPRISE PAR UN AUTRE DÉVELOPPEUR

Installer le projet en local

```
git clone https://github.com/hsidqui-dotcom/Networking-.git
cd Networking-
```

Aucune installation (pas de `npm install` pour l'app — c'est du statique).

Lancer l'application en local

Servir le dossier `app/` avec n'importe quel serveur statique :


```
cd app
python3 -m http.server 8000 # puis ouvrir http://localhost:8000
```

Note : pour tester la **connexion réelle** en local, ajouter `http://localhost:8000` dans **Supabase → Auth → URL Configuration → Redirect URLs**. Sinon l'app tourne en **mode démo** (données locales).

Tester les principales fonctionnalités

Suivre `tests/functional-checklist.md` (connexion, annuaire, programme, messagerie, partenaires, console...). Diagnostics rapides : `tests/diag.html` (moteur de connexion) et `tests/smoke.html` (latence). Charge : `tests/stress/` (k6, sur un projet **staging**, jamais la prod).

Faire une modification

1. Éditer les fichiers dans `app/` (frontend) ou `supabase/*.sql` (base).
2. Pour la base : exécuter le SQL dans **Supabase → SQL Editor**.
3. Tester en local (ci-dessus).

Déployer une nouvelle version

```
git add -A && git commit -m "ma modif" && git push
```

→ Sur la branche `claude/oneafricaforums-networking-app-hVvUm`, l'action GitHub **redéploie automatiquement** (≈ 1 min).

Astuce : à chaque évolution du frontend, **incrémenter** `const CACHE` dans `app/sw.js` (ex. `v52` → `v53`) pour forcer le rafraîchissement chez les utilisateurs.

8) ÉLÉMENTS CRITIQUES À SÉCURISER AVANT MISE EN PRODUCTION

#	Risque	Action recommandée	Priorité
1	Propriété des comptes : si Supabase/GitHub/Resend/Google/DNS sont sur des comptes personnels , l'entreprise peut perdre l'accès.	Supabase (org OneAfricaForums). À confirmer : dépôt GitHub rattaché à une org entreprise, Resend/Google/DNS au nom de l'entreprise + 2 admins.	.
2	~~Sauvegardes Supabase~~	RÉGLÉ — Supabase Pro : backups quotidiens + PITR 7 jours actifs. Garder un pg_dump avant/après événement.	.
3	Branche de déploiement au nom auto-généré (clauder/...).	Fusionner sur main et faire pointer pages.yml sur main (plus clair, plus robuste).	.
4	Dépendance CDN esm.sh : si esm.sh tombe, la connexion casse.	Héberger (vendoriser) supabase-js et qrcode dans le dépôt (et adapter la CSP).	.
5	Capacité temps réel	Pro actif (limite relevée). Pour un très grand événement, envisager l' add-on compute ; valider par stress test (prévu au bureau).	.
6	Limites Resend : à confirmer le quota d'envoi/jour du plan actuel.	Vérifier que le quota couvre le total d'invitations + codes des 2 forums le jour J ; sinon, plan adapté. Point critique avant l'événement.	.
7	Rotation des secrets (clé Resend, secret Google).	Documenter + savoir les régénérer (procédure dans §6).	.
8	LinkedIn désactivé (bouton masqué).	Configurer le provider quand prêt, puis réafficher (1 ligne dans app/index.html).	
9	CSP avec 'unsafe-inline' (gestionnaires onclick).	Acceptable ; durcissement = refactor (phase 2).	

9) ACCÈS PARTICIPANTS & DÉLIVRABILITÉ E-MAIL (stratégie de production)

L'accès à l'app **ne dépend jamais d'un seul canal**. Trois portes d'entrée, du plus fiable au moins fiable :

1. **QR code** (badges, accueil, programme, slides) → fiabilité maximale, aucun filtre.
2. **Lien partagé** <https://app.oneafricaforums.com/app/> (WhatsApp, LinkedIn, site) → arrive toujours.

3. **E-mail nominatif** (envoyé depuis la console) → confort ; une partie peut être filtrée par les serveurs d'entreprise (Microsoft 365 quarantaine = self-domain spoofing).
Non bloquant grâce aux canaux 1 et 2.

Kit prêt à diffuser : `docs/diffusion/` — QR haute résolution (`qr-oaf-connect.png/.pdf`), affiche A5 d'accueil (`affiche-accueil-A5.pdf`), textes WhatsApp/programme (`KIT-DIFFUSION.md`), demande IT (`DEMANDE-WHITELIST-IT.md`), ticket DMARC Genius (`TICKET-DMARC-GENIOUS.md`).

Renforcement délivrabilité (durable) : - **SPF + DKIM** : déjà en place et vérifiés sur `send.oneafricaforums.com` (chez Resend). - **DMARC** : à publier sur `oneafricaforums.com` via ticket Genius (mode `p=none` au départ, puis durcir `quarantine / reject` après l'événement). Valeur et procédure dans `docs/diffusion/TICKET-DMARC-GENIOUS.md` . -

Whitelist interne : règle à poser une fois par l'IT pour débloquer les destinataires `@oneafricaforums.com` . - **Réputation** : le domaine d'envoi étant récent (vérifié juin 2026), la délivrabilité s'améliore avec des envois réguliers.

10) CE QUE TU DOIS TÉLÉCHARGER, SAUVEGARDER, CONSERVER

1. **Le code** : GitHub → `Networking-` → **Download ZIP** (et/ou garder l'accès au dépôt).
2. **Un export de la base** : `pg_dump` (voir §5) — au moins **avant et après chaque événement**.
3. **Ce dossier** : `docs/LIVRABLE-TECHNIQUE.md` (+ `docs/AUDIT-2026-06-10.md`).
4. **Les accès** (§6) dans un **gestionnaire de mots de passe** au nom de l'entreprise.
5. **Les CSV sources** (`docs/programmes/`) et le **kit de diffusion** (`docs/diffusion/`).

Avec ces 5 éléments, le projet est **archivé, transmissible et indépendant** de tout environnement temporaire.